A Governed Supervisory Execution Layer for LLM-Agent Plant Writes

Anonymous Authors

***Abstract*—Large language model (LLM) agents increasingly propose control changes on industrial plants, yet the safeguards around each proposal—write authority, active limits, human approval, device acceptance—are rebuilt per deployment in application code, so a task reported complete can coexist with a missed process objective. The platform, an open-source governed supervisory execution layer, carries agent-originated writes along one path: admission against declared, versioned limits; approval where required; actuation over semantic write-control nodes; graded acknowledgment—readback-verified, transport-acknowledged, or unverified, with failed as a terminal failure state—by what a post-write read observes, not the transport receipt alone; and verdict recording with compensation—each stage a separately inspectable, actor-attributed evidence state. Team identities reach nodes only through explicit member–node capabilities, prompt content is never admissible as authority, and a fail-safe-manual recipe-dispatch master switch intercepts approval-free recipe dispatch. Four process scenarios—injection molding, wastewater treatment, annealing, and biaxially oriented polyethylene terephthalate (BOPET) film—plus a cast-film reference line exercise the layer across protocol-interception drills, agent-team missions, and a nine-run deployment-scale campaign. Across the archived runs, 33/33 out-of-window probes are rejected at the attributable layer, a publish-only endpoint that a success-flag rule would report as successful is graded unverified, and the campaign completes with zero failed assertions. The evaluation is simulation-tier; it establishes mechanism behavior and evidence integrity, not optimization benefit or safety assurance.**

***Index Terms*—Agent teams, human-in-the-loop, industrial informatics, LLM agents, supervisory control, write governance.**

I. Introduction

LARGE language model (LLM) agents are moving from advising industrial operations to acting on them: recent systems generate control updates from multi-modal input [1], pair each factory module with an LLM agent that dispatches skills over a tool protocol [2], allocate work among cooperating agents [3], [4], and act as real-time controllers of a robot line [5]. What these demonstrations leave implicit is everything that surrounds the proposal: who may change which parameter, which limits are active for the current product and recipe, whether a human approved, whether the device accepted the command, and whether the process improved.

Because these concerns live in application code, they are rebuilt per process, protocol, and agent framework—duplicated drivers, ad-hoc permission checks, logs untied to decisions. The four scenario plants—injection molding, wastewater treatment, annealing, and BOPET (biaxially oriented polyethylene terephthalate) film—differ, but all need an auditable bridge from a team-level task to heterogeneous acquisition and control nodes.

The platform—an open-source governed supervisory execution layer (Section III)—addresses this gap through member–node bindings (an engine-backed channel identity) and shared line, product, recipe, and run context. The response is organizational rather than algorithmic: each intervention leaves a separate evidence record per governed stage, from admission through compensation (Section IV); nodes expose engineering quantities, and one governed member–node binding path serves every client.

The primary contribution is a *framework*: one integrated design for agent-based industrial data acquisition, analysis, and parameter execution whose plant-facing writes are governed end to end. The central thesis is that the unit of assurance for an agent-originated write should be a graded, attributable evidence record rather than a success flag; Section V includes a case where the two disagree. Concretely, the platform makes four contributions:

1) a **capability-based industrial execution architecture** connecting agent-team identity, semantic data-acquisition (DAQ) and write-control (DCW, the platform's node-class tag) nodes, heterogeneous protocol drivers, and line–product–recipe–run context, with task membership distinct from physical-write authority and a default-manual recipe-dispatch master switch;
2) a **reusable integration boundary** of self-describing node and driver contracts across six protocol families (five instantiated as lines, four exercised for governed writes), versioned production objects, plugin extension points, and a typed software development kit (SDK); an onboarding contract projects scenario blueprints into it while process models and policies stay external;
3) an **evidence-preserving governed intervention workflow** separating admission, optional approval, actuation, graded write acknowledgment with post-write verification, verdict, compensation, and backstop—the grading rule stated falsifiably; and
4) a **layered evaluation** of one governed path: protocol closure with mechanism ablation, agent-team missions, a deterministic controller loop, concurrent process missions, and a deployment-scale regression campaign on a multi-line instance (Section V).

Scope, revisited in Section VI: real protocol stacks execute against author-implemented simulators; fast regulation, certified protection, physical-plant transfer, and optimizer ranking are outside the claims. The governed path is proposer-agnostic—prompt content is never authority and admission re-resolves per request—so the scripted proposers

TABLE I
CLOSEST LLM-BASED INDUSTRIAL CONTROL SYSTEMS VERSUS THE PLATFORM (VENUE, LLM ROLE, WRITE PATH, PRODUCTION-SCOPED AUTHORITY, WRITE EVIDENCE; ENTRIES SUMMARIZE WHAT EACH CITED PUBLICATION REPORTS, AND A MISSING MECHANISM IS NOT EVIDENCE OF ABSENCE).

System (venue)	Role of the LLM	Plant write path	Authority & production context	Write evidence & verification
MADE, ICCAD 2026 [1]	multi-modal control-update generation	generated commands + safety layer	HITL verification of outputs; task-level checks	task success over scripted evaluations
Köhle et al., ETFA 2026 [2]	per-module agents, MCP skill dispatch	skill invocation via OPC UA/MQTT	module-scoped skills	task-resolution rate (6-module plant)
Lim & Kovalenko, CASE 2025 [3]	capability exploration across agents	architecture-level	resource-matching scope	architecture evaluation
Zhao et al., AEI 2026 [4]	shop-floor task dispatch	workshop experiments	coordination layer	dispatch/execution performance
Waseem et al., Mach. Learn. Eng. 2026 [5]	real-time controller in the loop	direct loop actuation (robot line)	prompt-level constraints	line KPIs under LLM control
The platform	**proposer (replaceable strategy slot)**	**governed write path: admission, HITL, dispatch, readback**	**member–node bindings; recipe-dispatch master switch; product/recipe-scoped limits**	**graded acknowledgment; evaluation-harness-checked three-way consistency; attributed journal; compensation; backstop**

of Section V exercise the same contract an engine-issued proposal would face.

II. RELATED WORK

A. LLM Agents in Industrial Control: From Generation to Governance

Agent-based manufacturing and supervision have long linked organizational roles to plant capabilities [6], and LLM-oriented industrial work now spans PLC generation and verification [7] and direct plant-facing action—context-aware control [8], digital-twin fault handling [9], and natural-language machine control over OPC UA [10]—without production-scoped admission, per-request re-checking, or graded write evidence. The closest systems (Table I) target the control action itself—multi-modal control-update generation behind a safety layer [1], module-scoped skill dispatch over a six-module plant [2], and pretrained LLMs *inside* the control loop as real-time controllers [5], a layer below the platform so the roles are complementary—while agent-allocation work [3], [4] distributes tasks, not governed writes. A recent review frames agentic AI's industrial value as safety-aware human-in-the-loop (HITL) decision support rather than autonomous control authority [11]—the mechanism set here makes that concrete.

B. Frameworks, Standards, Run-Time Guarantees, and Automation Practice

The Model Context Protocol (MCP) standardizes multi-agent tool interfaces [12]; its tools carry advisory annotations but no production context—line, product, or recipe—and no record separating admission from execution. ISA-88 and ISA-95 model recipes and operations [13], and the platform's versioned recipes instantiate a reduced subset of that stack; the governed life cycle transposes run-to-run (R2R) control disciplines [14] onto a supervisory write path. A separate line enforces run-time guarantees on LLM agents—schema-level behavioral constraints (AgentSpec [15]), and capability-based information-flow control (CaMeL [16]), the closest software-agent analogue of the rule that prompt content is never authority; they gate software tool calls and data flow, not plant writes; their policies could compile onto this substrate.

Permissions, range checks, approvals, and audit trails long existed in distributed-control-system (DCS) and SCADA practice—versioned batch recipes and electronic batch journals under 21 CFR Part 11 [17], second-channel monitoring and run-to-run setpoint adaptation—while IEC 62443 codifies their security counterpart [18] and formal shielding, control-barrier functions, and simplex run-time assurance offer stronger properties [19]–[21]. Graded command confirmation itself has telecontrol ancestry: DNP3's select-before-operate exchange [22] confirms a command at the protocol and application layer; the platform grades above it, against production context and actor attribution. Against that practice the difference is the proposer: a machine-generated agent identity rather than a presupposed human operator, so admission binds to member authority, prompt content stays inert, and intervals are re-evaluated per request against live product and recipe state. No safeguard above is claimed as an invention—what is new is their composition into this one proposer-agnostic path, whose evidence states—graded acknowledgment, verdict, compensation—survive strategy replacement; the three-way check is a supervised special case of classical data reconciliation. The platform implements source-dependent checks and a rule-based backstop (Section IV-D), no formal protection guarantee; human oversight likewise requires more than an approval dialog [23].

III. SYSTEM ARCHITECTURE

The platform occupies the replaceable strategy slot that Section II's comparison leaves open: it links team coordination to plant interaction through industrial nodes and production context (Fig. 1). Responsibilities group into five planes—logical groupings, not deployable units, each an assembly of modules whose outputs are evidence records: **team** (channel and member identity, task state, episodic memory, engine adapters, retrieval, analysis) turns acquired data into attributed proposals; **integration** (DAQ and DCW nodes, per-node runtimes, protocol drivers, gateway, MES mirror/direct modes) owns protocol acceptance, readback, freshness, and transport errors, presenting devices as semantic nodes; **governance** (binding authority, admission resolver, approval queue, recipe dispatch, acknowledgment grader and verifier, verdict, compensation, backstop, journal) owns the governed life cycle; **operations** (line, product, recipe, and run registries, versioned recipes, run ledger, operations log) attaches production context while PLC protection stays

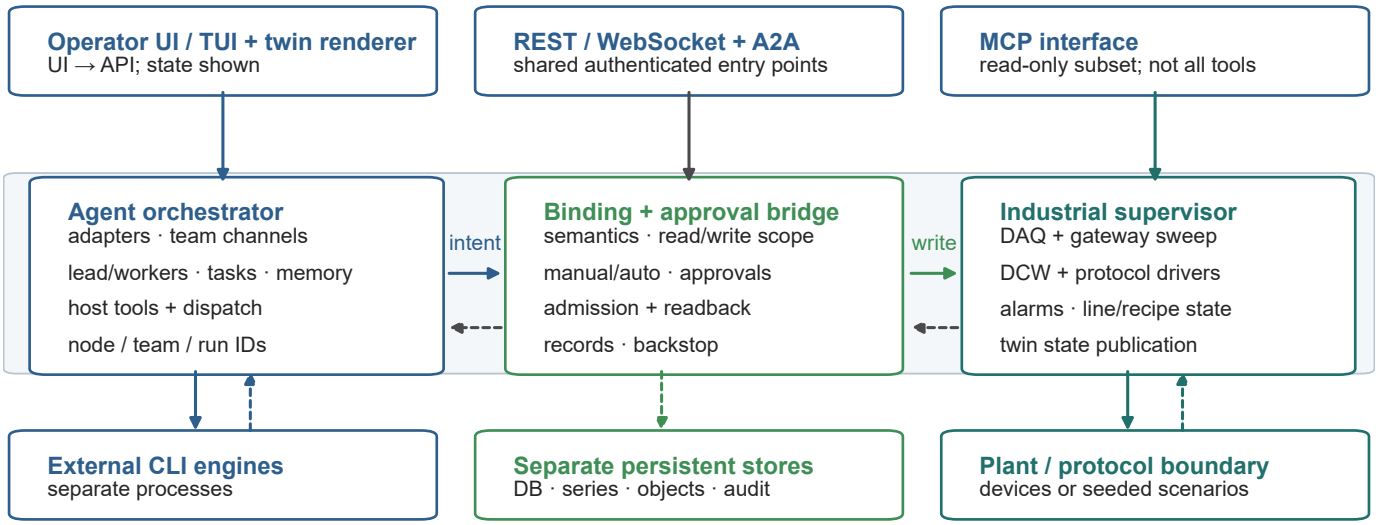


Fig. 1. System architecture and execution boundaries: platform services around shared stores, with agent engines, data stores, protocol endpoints, and the plant kept separate. Shared identifiers provide attribution but do not make storage and actuation transactional. Solid arrows: request/control; dashed: observation/record; PLC safety and hard real-time control remain external.

independent; and **extensibility** (plugin registry, typed SDK, REST/MCP/A2A/CLI/TUI entry points) admits drivers, tools, and clients through sanctioned registration services. Scenario knowledge sits in blueprints, signal maps, predicates, and driver configuration—not in scheduler, approval, or persistence code; members are interchangeable across engine adapters, and team topology, bindings, and limits are data rather than code (Fig. 1). The subsections below describe the node classes, the binding boundary, and the task life cycle; Section IV specifies the mechanisms the governance plane implements.

As an open-source project under the Apache-2.0 license (links withheld for double-blind review), the platform ships these planes as one service with 58 industrial agent tools, the typed SDK, and the one-command benchmark pipeline of Section V-B.

A. Supervisory Core and Nodes

The Node.js core hosts channel routing, industrial services, and protocol adapters. A channel contains a lead, workers, tasks, messages, and retrievable memory, deployed through WebSocket, MCP, agent-to-agent (A2A), REST, CLI, or TUI entry points; channel membership alone confers no write authority.

DAQ nodes convert device readings to engineering quantities across four modalities—scalars, vector profiles, image frames carrying content fingerprints, and mirrored manufacturing-execution-system (MES) records—so plant data-shape heterogeneity does not reach the team loop. MES endpoints are reached in *mirror* mode, pulling records through the same acquisition pipeline, or in *direct* mode, declaring read and write mappings against the MES interface. DCW nodes translate engineering setpoints into driver commands and, where supported, readback; per-node runtimes manage timing, and gateway services supply connection state, recipe context, and write bookkeeping. The acquisition surface declares six protocol families, five with drivers today—Modbus TCP, Modbus RTU-over-TCP, OPC UA, MQTT, HTTP/REST, and an S7 stub that fails loudly until its driver lands; the write surface adds one driver per writable family plus MES REST (mock drivers excluded), and the evaluation instantiates five families as lines (Section V-C). The deployment reaches author-implemented dynamic simulators through the same node and driver contracts a plant would use; they are deterministic per seed, not validated digital twins. Reuse is declarative—signals, units, scaling, precision, driver configuration—and the team loop is unchanged.

Driver contracts are deliberately narrow: a driver declares how a semantic quantity maps to its transport payload, whether the endpoint is writable, and whether a readback is available; the controller resolves authority and active limits. Errors attach to the request, register addresses and byte order appear only here, and a new fieldbus family leaves the loop untouched; undeclared drivers fail loudly rather than being silently mocked. Acquisition is supervisory rather than hard real-time: each DAQ node runs its own cadence under a rotating sweep; the verdict stage queries the timestamped, quality-stamped history, not a blocking read.

B. Bindings, Context, and Reuse Boundaries

A member–node binding grants access to a sanctioned tool on a specific node and selects automatic or approval-mediated execution. Line permissions (none, read-only, operate) authorize binding creation and mutation; runtime invocation is authorized by the persisted capability, not channel membership. Intervals are re-checked every request, so revoking a binding or switching the recipe takes effect on the next request; the same path serves a console, a harness, and an agent engine—authority does not depend on the client, only the evidence record differs.

Production objects are versioned rather than overwritten: recipes dispatch against a known-good version, revert non-destructively, and are compared with the values actually applied; an intervention is read against the definition active when it was admitted. Plugins register hooks, routes, agent

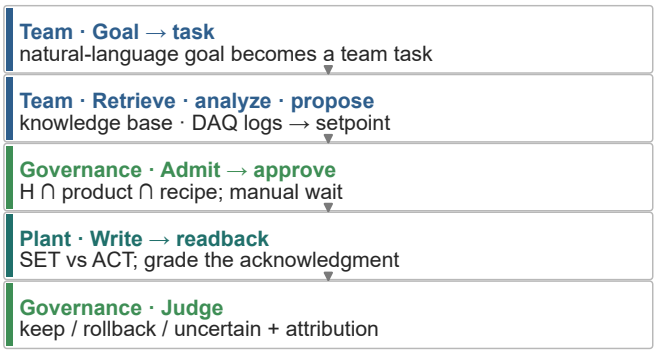


Fig. 2. Implemented agent-team runtime. A natural-language goal enters as a team task; retrieval, analysis, and the lead produce a setpoint; the governance stages admit the value against Eq. (1) and, on a manual binding, wait for approval; the write path issues the command and reads back the actual value (SET versus ACT); the judge records keep, rollback, or uncertain with attribution. Governance rows are deterministic platform logic, not model-bearing agents.

tools, drivers, and panels only through sanctioned services, so an added driver inherits admission, readback, and evidence capture. Scenario reuse is bounded by an onboarding contract: the engineer declares a blueprint (devices, signals, units, roles, setpoints); a differential "ensure" pass creates missing devices, repairs drift, and projects the blueprint into semantic nodes and a recipe; a channel is built from the team template, and a mission supplies objective, guards, response windows, and policy. A BOPET worker proposes casting-speed, fast-roll, or rail-width changes through bound DCW nodes; the other three scenarios reuse the same pattern (Section V-F).

C. Governed Optimization Task Life Cycle

A governed optimization task is a life cycle rather than a single tool call, and the platform exposes each transition separately. The task enters a channel carrying goal, line, product, recipe, run, and task identifiers; a lead member claims it and hands it to a worker holding member–node bindings, whose fixed job repeats: read the controlled quantity, propose a setpoint inside the resolved interval, invoke the governed write, and record a verdict (Section V-B instruments it).

Each iteration leaves an optimization record, the acknowledgment grade, an attributed journal entry, predicate attainment, and a workflow terminal state. Two write lifecycles coexist: direct node writes (agent `dcw_control` or admin REST) open these records, while recipe-chain operations (batch dispatch) are gated by the run ledger and open none. This bounds what "automatic optimization" means: a controller, scripted policy, optimizer, or agent engine may occupy the strategy slot, while authority, evidence, and closure stay on the platform.

IV. Governed Execution Mechanisms

A supervisory intervention must preserve distinctions that an agent tool call usually collapses—permission to propose, admission of a value, driver acceptance, device confirmation, process response, a caller's verdict, and compensation—and the platform exposes these as separate evidence states. Three design commitments generate that separation: model output is an untrusted proposal, never authority; evidence is graded by what is observed, not claimed; and failure states surface for inspection rather than being absorbed into a success flag. Fig. 2 shows the implemented agent-team runtime carrying these steps. A governed intervention is one transaction in the audit-trail sense: every step leaves its own evidence record, while execution and recovery are deliberately not atomic with device state (Section VI). Six invariants organize the path—binding-resolved authority (never channel membership or prompt content), Eq. (1) admission, fail-closed approval waits on manual bindings, graded rather than success-flag acknowledgment, checkable three-way consistency, and separately admitted compensation—each specified below; every agent-originated path to the plant passes through the same governed write.

A. Task, Observation, and Admission

The strategy receives a task and goal in production context and decides what to propose and when to stop; the platform governs semantic node parameters—pressure, dosing rate, line speed, rail width—in engineering units.

For an ordinary agent setpoint request on node n, the admission interval is

$$\mathcal{I}_n(t) = \mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t) \cap \mathcal{R}_n(t), \tag{1}$$

where $\mathcal{H}_n$ is the mandatory hard engineering range (finite and non-empty at node creation) and the remaining terms the baseline, active-product, and active-recipe limits (together the soft window); a missing optional limit contributes no restriction, and t is the admission instant, re-resolved at execution. A configuration whose admitted interval is empty rejects every write, naming the excluding layer on the first attempt. The interval form is deliberate: every bound is a declared, versioned object an audit can replay against active product and recipe; rejection is decidable and layer-attributable (Fig. 3a)—decidability, not a safety certificate. It bounds one parameter at a time—joint windows live in scenario guards—and bounds magnitude, not intent: availability, binding state, and approval policy are checked separately, and in-window but adverse sequences surface at verdict and compensation (Section V-F reports two).

B. Semantic Write, Pacing, and Source-Specific Routes

A semantic card exposes quantity, unit, precision, range, value, freshness, and recipe context; the controller resolves binding and limits, the driver serializes the command, and the acknowledgment grade closes the command path, never the process-response path (Section IV-C). Three pacing windows pace governed loops, never the plant: online writes *on a node*—agent- or human-origin—sit at least 60 s apart with a hold window after success (HTTP 429 inside it); a per-recipe dispatch interval anchored at the *approval* instant is restart-persistent, so alternating identities cannot reset it; and a 300 s per-line trial cadence damps same-line trials from any origin; recipe-dispatch and rollback writes are exempt. `AW_BENCH_MODE=1` admits harness-declared arms that relax the soft window or readback—never the hard range.

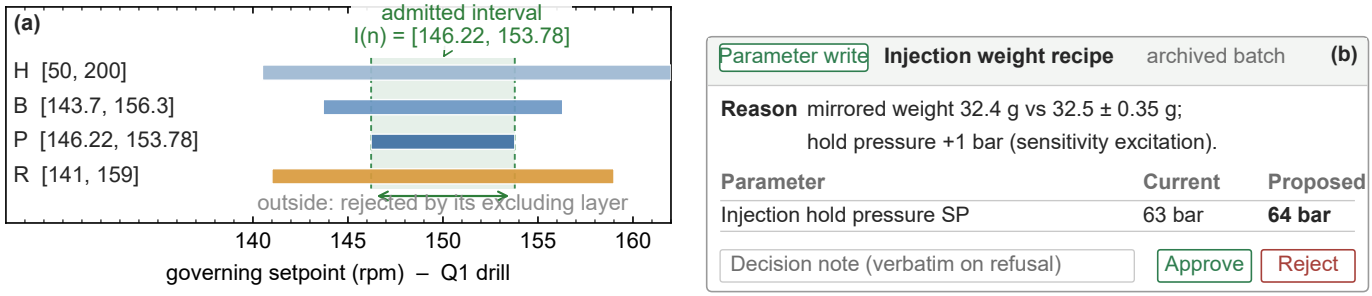


Fig. 3. The admission boundary and its human-approval gate. (a) The Eq. (1) interval geometry: four declared, versioned bounds narrow to the admitted interval; a proposal outside it is rejected by its excluding layer (the Q1 drill). (b) The structured approval card on a manual binding (English vector redraw of the live console). One setpoint—current 63 bar, proposed 64 bar—is separated from the proposer's display-only reasoning; the decision note returns to the member on refusal (Section IV-E).

Four governed routes—agent tools, human-origin writes, recipe dispatch, rollback compensation—reach one admission and journaling entry point, the hard range included; recipe dispatch adds run context—the line-scoped execution window, the reduced ISA-88 batch/lot notion—validated at save time. Hold heartbeats (periodic stored-setpoint re-dispatch) are the one non-governed re-dispatch: off by default, stopped by line stop or driver change, and re-applying the stored value without re-admission—plant-facing state maintenance, not an agent path. Line start installs the active context before recipe parameters dispatch individually; line stop clears it.

C. Write Acknowledgment as Graded Evidence

A driver-level success flag conflates two facts: the transport accepted the command, and the device now holds the value. The platform grades every governed write at two checkpoints. At *acceptance*, the driver records a provisional class: *readback-verified* (its own post-command read returns the commanded value within the node's configured write tolerance), *transport-acknowledged* (the transport accepted the command, nothing device-side observed yet—a broker receipt or an MES business acknowledgment), or *unverified* (the transport failed). At *verification*, a post-write verifier re-reads the endpoint through the node's read path (three attempts, 2 s apart) and promotes transport-acknowledged parameters whose read returns the commanded value; a write that cannot be confirmed closes *unverified*, and a command that never completed acceptance closes *failed*, overriding the provisional unverified. The terminal grade is fixed when the write closes; the per-write journal preserves the acceptance class and the terminal grade. We therefore distinguish three evidence levels—readback-verified, transport-acknowledged, and unverified—plus one terminal failure state, *failed*. Batch receipts carry a per-parameter terminal grade and a three-bucket summary (verified/unverified/failed); the summary counts terminal outcomes, while transport-acknowledged is a per-write terminal grade on acceptance-receipt routes, not a batch bucket. The operations log records worst-state severity. The falsifiable contrast: under a success-flag receipt rule—a transport acknowledgment closes the report—the publish-only receipt of Section V-G (broker puback, `ok=true`, no read channel) reports *success*; the graded rule reports it *unverified*. Two boundary cases fix what the grades mean: a MES write whose endpoint returns an explicit application-level acceptance but exposes no readback mapping terminates transport-acknowledged—an acceptance receipt, still short of device confirmation—whereas a broker receipt alone terminates unverified. Absence of evidence degrades the grade, never the report; the grade is fixed when the write closes.

A transport-acknowledged write may still not have reached the device; promotion through the verifier turns acceptance into confirmation, and the three-way check of Section IV-D re-reads the value from the acquisition store. Recipe rollback restores a definition, not physical PLC values, and a compensating write is graded by the same rule, readback-verified where the endpoint supports readback and at best transport-acknowledged where only an acceptance receipt exists. The grade closes the command path only: readback-verified says the device holds the value, not that the process responded as intended—that distinction belongs to observation and verdict.

D. Observation, Verdict, and Compensation

An agent- or rollback-source write opens an optimization record; the verdict tool `dcw_judge` records `keep`, `rollback`, or `uncertain` plus a reason; the verdict, stored with the record it closes and the window queried, performs no actuation and does not independently validate the evidence, so a scripted keep can coexist with a missed objective. Verdict integrity therefore rests on attribution, journaling, and operator review—a kept-but-marked record stays visible with its disagreement attached, and the operations log raises parameter severity under alarm-rationalization practice—with the next guard queued: an independent adjudication identity, or a platform-side minimum re-check before a caller's `keep` may stand down the backstop (Section VI). Three-way setpoint consistency is checked by the evaluation harness rather than the platform: the harness compares the recipe value, the device readback, and a mirror sample drawn from the acquisition runtime's separate polling path and store, each leg against a stated tolerance; the sampled legs meet at the plant endpoint, so a fault fabricating the endpoint is their common failure mode, and disagreement is reported with the run. Verdicts are attributable—actor and timestamp travel with them.

Compensation is a new, separately admitted write that may fail or partially complete; it is graded by the Section IV-C rule, and a rollback record settles to rolled back once the

compensating write is accepted. If the compensating write is itself refused by admission, the source record closes *as judged*, with the refusal appended to its verdict. These separations keep command status, record status, and process recovery independently inspectable.

The backstop is a record-integrity check for writes that no verdict has closed—not an independent protection layer: it evaluates only open records; a caller's `keep` closes and stands it down, while a `rollback` verdict stands it down only once the compensating write executes—availability is therefore contingent on recorded verdicts and arriving observations. When collected samples of an open record's node leave the recipe's monitoring window (a DAQ-side envelope distinct from the admitting interval and from the mission's attainment window), the platform issues a system-attributed rollback verdict and restores the value preceding that write (one step; the admitted baseline is not restored), graded as above. The policy waits 120 s, polls every 30 s, and fires on three cumulative out-of-window samples; non-returning samples count no breach, so an acquisition outage leaves the record open and unalarmed. Recovery runs only while fewer than two retained rollback records stand on the node; past that, the backstop judges but defers actuation to the approval policy.

E. Human Approval Boundary and the Line Master Switch

Manual bindings insert the timed approval wait before an agent setpoint or compensation request; automatic bindings omit the wait but keep the checks. The pending request carries member, node, value, interval, expiry, the proposer's reasoning, and per-parameter bases; Fig. 3b shows the card's value row, reasoning line, and decision box—machine-checked fields as structured data, the free text display-only and non-authoritative. Because binding and admission are re-checked at execution, approval reserves neither interval nor device state; the approval dialog—itself an injection surface (Section VI)— cannot promote a proposal past the checks; the comment returns verbatim to the proposing member on refusal and travels with the receipt on approval—feedback, not authority. The wait fails closed by default (expiry 180 s): an expired request resolves as refusal and is recorded; duplicates are suppressed by holding one pending request per member–node pair; and in *hold* mode an unanswerable request persists without expiry until a human answers.

Human authority is anchored one level above individual requests: each line carries a control mode—the recipe-dispatch master switch—whose fail-safe default is *manual*. Under manual mode, recipe-family operations (parameter update, dispatch, trial, and rollback) from approval-free bindings on that line are intercepted and presented for approval regardless of binding policy; direct node-tool writes remain gated by their binding mode and the admission interval, not by the line switch. Switching a line to automatic requires an explicit admin-only user-side confirmation; agent identities cannot touch the switch. The residual agent-side path is delegation: a lead agent can grant held nodes to a worker through an attributed lead-only tool; the grant's execution mode is chosen at grant time rather than gated by the user-side confirmation, is attributed to the granting agent, cannot exceed its own node face, and is individually revocable (Section VI).

The decision endpoint authenticates a user but does not enforce a line-scoped approver role—the gap Section VI carries—and reads, verdict recording, and some configuration changes do not use physical-write approval—workflow mediation, not segregation of duties; a recipe switch between approval and execution re-admits the value against the new window. The engine is replaceable, not a control primitive—scenario code declares and configures but never executes plant I/O, and the platform never infers a process objective.

V. Experimental Evaluation

The evaluation asks whether one governed supervisory path can be exercised, inspected, and repeated across heterogeneous industrial jobs, not whether one optimizer beats another. Five questions organize the evidence: protocol closure and admission-gate attribution (Q1), the agent-team workflow (Q2), a deterministic controller loop (Q3), four concurrent missions (Q4), and deployment-scale regression with write-evidence verification (Q5). Each question is reported as procedure and archived result. Q2–Q4 occupy the strategy slot (scripted policies, with two live engine turns in Q2); Q5 stresses the hardened platform under load.

A. Testbed, Evidence Snapshot, and Scope

The evaluation draws on two tiers of one codebase. The *archived snapshot* for Q1–Q4 is an integrated run (seed 42) with raw reports and an ablation companion run. Its standing preset is a cast-film reference line tuned through the die-head heater setpoint (objective J: thickness conformance, defects, energy, throughput). Each mission contributes one seed-42 trajectory: Q1, Q2, and Q4 are single-trace case studies; the deterministic loop adds seeds 43–44, and repeat evidence comes from Q5's deployment-tier campaigns—a different tier, not a re-run.

The *deployment-scale tier* for Q5 is a live instance carrying 34 provisioned lines, 111–113 agent channels, 67 online DAQ nodes, 77 write nodes, 52 recipes, and 385 member–node bindings—simulator-backed, not physical plants; *multi-line* means many provisioned lines and channels on one host. The tier's benchmark line—distinct from the cast-film reference—is a physics-based injection-molding cell (16 DAQ nodes, 6 control nodes over five protocol families plus MES, four exercised for governed writes); all Q5 numbers come from archived per-run reports and timelines (three passes of three runs each), with 17 of those lines running and 111–113 agent channels active. One timeline fact matters: acknowledgment grading and the master switch came with the later hardening pass, *after* the Q1–Q4 snapshot—Q1–Q4 establish the governed path without them, and Q5 evidences both surfaces; the ablation isolates the window interlock, untouched by that pass. The harness sets that line's recipe-dispatch master switch to *automatic* by user-side confirmation—the automated path is the object under test—and verifies manual-mode interception separately (legs 1 and 5).

B. Test Procedure and Reported Quantities

Evidence unit. Every execution emits per-assertion check records, phase records, per-line metrics, a timestamped timeline, and a rendered report; a claim below is read from those artifacts, and a run passes only when every check in its declared surface passes.

Q1–Q4 procedure. One integrated suite runs 20 named phases in fixed order, each owning one claim: P0–P2 commission the testbed (simulator bootstrap; preset with an offline optimum W^*; up to five protocol lines); P3–P4 exercise acquisition and the write path (DAQ through real drivers; governed write and readback; window-edge interception; a tool-level loop without an LLM); P4b–P4f exercise the governance surfaces (records and rollback, HITL, audit, recipe life cycle, parameter mapping); P4m, P5, P5b, and P10 run missions (scripted team task; two live engine turns and a recipe save; the multi-node BOPET mission); P6–P8b close the loop and test reuse (three-seed controller benchmark, vector and image frames, a configuration-only second preset, the backstop drill); P9 and P11 check subsystems and four concurrent missions. The suite is an author-defined regression instrument, not an external certification.

Q5 procedure. The deployment tier runs the one-command pipeline: S0 pre-check; S1 API surface; S2 multimodal ingestion with MES dual-mode; S3 governance negative paths; S4 closed loop; S5 fine-tuning; S6 diagnostics; S7 report. Three passes of three runs each (nine executions) replay the same surface, and a seven-leg suite (Section V-G) covers write-evidence and regression surfaces.

Controls and quantities. The admission control relaxes the mechanisms: the six window-edge probes that full governance rejects (HTTP 400) return 200 *executed* under the relaxed arm (Table II)—a success-flag design reports those writes as accepted. The evidence control holds the receipt and varies the reporting rule (Section IV-C): applied to the same publish-only dispatch, a success-flag rule reports success while the graded rule reports unverified (Section V-G, leg 4). Reported quantities are interception (rejected probes over probes issued, layer named), acknowledgment (terminal grade per parameter; batch summary), consistency (maximum deviation among recipe, readback, and mirror), attainment (signed margin, decided by an archived check), latency (server-side span with sample size), and repeatability (checks passed over checks run); with $n \approx 6$ writes per line, distributions are medians with observed maxima, not percentiles. Fig. 4 summarizes the closed-loop instrument.

C. Q1: Protocol Closure and Admission-Gate Attribution

Protocol closure. On the five protocol lines the suite stores 55 DAQ samples through real drivers and dispatches governed writes over four protocol families—about 6 writes per line, with median command latency 19.3–47.8 ms and observed maximum 20.1–248.9 ms. The RTU-over-TCP satellite is acquisition-only—readback confirmation where supported—with one admission-record-verdict path per request. Each writable line rejects 6/6 out-of-window probes (24/24) while accepting its legal write.

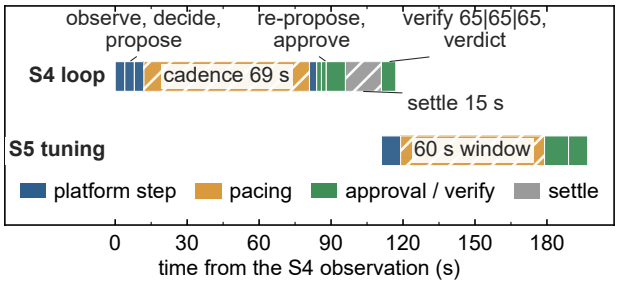


Fig. 4. One governed iteration on the deployment line. Top lane (S4, closed loop): a corrective decision (0.276 g error) is packaged as a sensitivity excitation; the trial cadence early-rejects it 69 s before a human is involved; structured approval releases the re-proposal, dispatch returns readback-verified, and the three-way check reads 65—65—65. Lower lane (S5, fine-tuning): the same 60 s window, rolled back and restored.

TABLE II
ADMISSION ABLATION ON THE COMPANION RUN: SIX PROBE CLASSES PER ARM, THREE DETERMINISTIC REPETITIONS. THE FOUR HARD-RANGE CLASSES ARE REJECTED IN EVERY ARM; ONLY THE TWO WINDOW-EDGE CLASSES FLIP. "JOUR." = JOURNALED.

Arm	Soft win.	Readback	Intercept	Executed
full	✓	✓	18/18	0
no-readback	✓	—	18/18	0
no-interlock	—	✓	12/18	6 jour.
ungated	—	—	12/18	6 jour.

Drills. Two drills separate truth paths one success flag would collapse: a frozen-measurement drill (write and readback consistent, measurement out of window) ends with a system-judged backstop rollback 130.2 s later, and a record-rollback drill restores the prior value by a separately admitted, readback-confirmed write. A second preset with different signals and dynamics is commissioned by configuration alone; across both presets 33 probes are rejected on seven writable lines (24+9)—the same admission code discriminating on unseen signal maps. The parameter layer makes the interval geometry of Eq. (1) observable (Fig. 3a); a manual binding suspends a request visibly and unblocks it on approval (latency 461 ms, harness-adjudicated).

Ablation. The ablation replays the same probes on a companion run and a standalone fixture (six probe classes per arm, three deterministic repetitions). Table II reports four arms: with the interlock, 18/18 probes are rejected with or without readback; without it, only 12/18 reject and the six window-class probes execute and are journaled—prevention is the interlock's job, and the readback check only detects. The hard range is structurally unbypassable—all four governed routes re-admit it at one journaling entry point, per request (Section IV-B)—and the soft window is its removable part.

D. Q2: The Agent-Team Optimization Workflow and Its Results

Missions. The workflow ran end to end on the cast-film line and then a larger BOPET line (Section III-B); a scripted policy issued the writes. On the single-node mission one write sufficed ($200 \rightarrow 204.704\,^{\circ}\mathrm{C}$ against target $204.704 \pm 0.75\,^{\circ}\mathrm{C}$, settled at $204.70\,^{\circ}\mathrm{C}$, journal-attributed, task `COMPLETED`). The BOPET mission is the multi-node case: nine devices and

49 signals (30 writable, 19 readable), all ensure-pass created. Three governed writes on three distinct nodes—casting speed $32 \rightarrow 33.8$ m/min, fast-roll speed $118 \rightarrow 120.3$ m/min, and rail width $3000 \rightarrow 3035$ mm— move the archived thickness trajectory from 28.10 to 25.66 μm against $25.0 \pm 0.7\,\mu$m, $0.04\,\mu$m inside the upper bound. The structured attainment check evaluates $|25.66 - 25.0| = 0.66 \leq 0.7$ and passes, as the regenerated summary now reports; the $0.04\,\mu$m margin is four $0.01\,\mu$m quantization steps, a boundary pass rather than a robust process win. Each write opened a record, received `keep`, and left an agent-attributed journal entry; the parent task reached `COMPLETED`.

Live engine turns. Two engine turns (omp adapter, glm-5.3-flash) drove the same slot on a scratch line—a bounded write (156.3 s), a goal-driven descent settling at 51.7 μm against a harness-supplied target ($52.0 \pm 0.8\,\mu$m, in the task text), and a recipe save with attributed authorship (637.3 s). They show an engine occupying the slot, not agent quality—the run's pass criterion is a completion marker in the transcript.

E. Q3: The Deterministic Closed Loop

Closed loop. Seeds 42–44 each issue two governed writes without rejection, raising the cast-film objective J from $J_0 \approx 67.2$ toward its grid optimum ($J/J^* = 0.965$–0.972 against $J^* = 89.894$; the grid enumerates the admissible zone-temperature, screw-speed, and line-speed setpoints, so J^* lower-bounds the true optimum and the ratio upper-bounds J/J_{true}). The claim is narrow: the governed path passes a conventional controller's writes through unmodified—admissible, journaled, read back—so this evidences transparency of handling, not a performance delta.

F. Q4: Four Process Missions

Four missions. The campaign runs four blueprint-provisioned lines concurrently; in this run (seed 42) the archived structured checks evaluate true for all four, from subsequent observations rather than task state, and two are boundary passes (Fig. 5). Injection molding is the minimal case: two writes (hold pressure 45→58.2 bar, hold time 6→10.64 s) move part weight from 31.25 to 32.17 g against a 32.5 ± 0.35 g window, a boundary pass 0.02 g above the lower bound; BOPET is the multi-node case of Section V-D. Both close with task state and predicate agreeing. Wastewater treatment (anaerobic–anoxic–oxic) is the write-heaviest mission (15 governed writes across distinct nodes—the Section IV-B spacing is per node): dissolved oxygen rises from 0.21 to 3.50 mg/L, inside the 3.4 ± 0.6 mg/L attainment window (Fig. 5b). All five effluent guards remain inside their hard limits; four finish with a wider margin than at baseline, while pH narrows from 6.59 to 6.38 (limit 6.0). Attainment is therefore on the controlled DO quantity, not on guard margin. Its most informative events are blower steps that were admissible and journaled yet lost compliance margin; the verdict stage reverted them—a bad proposal is rejected in the evidence, not at the admission gate. Annealing starts far outside the hardness window (134.52 HV at 140 m/min); zone-3 temperature, one zone-2 trim, and two line-speed steps hold 95 ± 6 HV (terminal 100.08 HV, 0.92 HV inside the upper bound) while raising line speed by 17.1%, and a further attempt at 176 m/min is reverted for leaving the margin—on record. Mission clocks (26.48 s, 169.27 s, 102.76 s; BOPET's unarchived) are simulated durations under accelerated dynamics; platform behavior is identical across the four while the number of writes varies sevenfold (2, 15, 5, and 3).

TABLE III
CAMPAIGN PROGRESSION ACROSS THE THREE PASSES; EVERY EXECUTION COMPLETES WITH ZERO FAILURES.

Executions	Test surface	Per run	Total
Pass 1 (3 runs)	baseline S0–S7 suite	79	237
Pass 2 (3 runs)	+ grading, master-switch checks	90–92	273
Pass 3 (3 runs)	same surface under live load	90–92	272
nine executions	zero failed checks	—	782

G. Q5: Deployment-Scale Regression and Write-Evidence Campaign

The final lens stresses the platform: the hardening pass made evidence grading, the master switch, and the approval-anchored dispatch interval of Section IV explicit test surfaces of the closed-loop, fine-tuning, and write-evidence scenarios. *Full-suite repeatability.* Nine pipeline executions complete with zero failed checks (Table III); the first two passes wait through the remaining 218 s of the per-line trial cadence—pacing stays in force under automation.

Closed-loop scenario under governance. Each run walks observation, decision (dead-band 0.15 g, slope 0.052 g/bar), a five-element proposal, HITL approval, batch dispatch, and three-way consistency (recipe, PLC readback, mirror). The first two passes descend $66 \rightarrow 63$ bar one governed step per run; the third pass answers a downward drift in the mirrored weight ($32.49/32.33/32.22$ g) with a bounded zigzag (64→63 bar), deliberate excitation rather than a correction, and two corrective steps (63→65 bar) answering residual error ($0.169/0.276$ g). Every step verifies equal values across the three sources at maximum consistency error 0.0 bar (0.05 bar device, 0.51 bar bucketed-mirror tolerances—all legs equal at the endpoint's recording resolution). The pacing window acts before the human: in two third-pass runs the proposal is early-rejected by the trial cadence and re-proposed 69 s later; only then is the HITL card raised and structured approval releases the readback-verified dispatch (Fig. 4). Approval-to-dispatch takes 6–9 ms; observation-to-verification spans 25.1 s with no pending cadence window and 96.1 s (third pass) or 245.20/245.18 s (first pass) with one—the loop's latency is governance, not platform overhead. The fine-tuning scenario rides the same windows; the diagnostics scenario exports a 30-min wide table (663–869 rows), a data-quality observation rather than a process result.

Latency figures are single-run point estimates: the Q1 approval latency is the harness's mediation span, not a human response, and Q5's approval-to-dispatch span is server-side. Only Q3 repeats across seeds, Q5 across consecutive campaigns, and the Q1 ablation deterministically (Table II); the Q1–Q4 missions and the fault-suite grade outcomes (n=1–2) remain single-trace case studies.

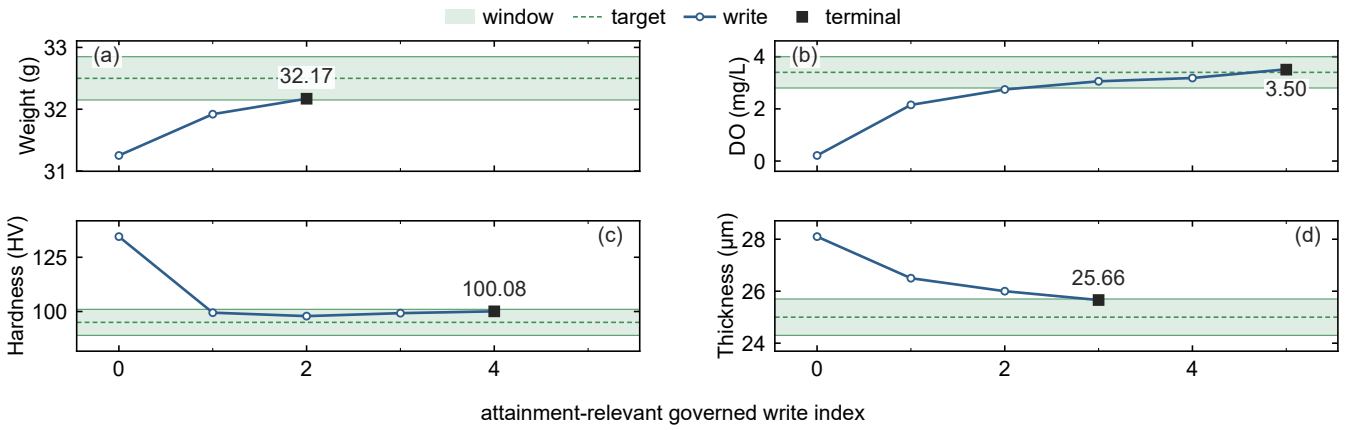


Fig. 5. Archived optimization trajectories of the four concurrent missions (seed 42): observed controlled quantity after each attainment-relevant governed write, against the archived objective window; the index counts those writes ((b) plots 6 of its 15). (a) Injection molding; (b) wastewater treatment (A2O); (c) annealing; (d) BOPET film. Panels show part weight, dissolved oxygen, hardness, and thickness; observations are from scripted policies.

Write-evidence legs. A dedicated seven-leg suite verifies Sections IV-C and IV-E directly in legs (1)–(5): (1) the recipe-dispatch master switch refuses an unconfirmed manual-to-automatic transition; (2) a mock-acknowledged (transport-acknowledged) dispatch is promoted to readback-verified by post-write re-read; (3) a broker cut marks the affected parameter *failed* at the batch summary while its healthy sibling verifies; (4) a publish-only endpoint accepts the write (broker puback, `ok=true`) yet stays *unverified* through three verification attempts; applied to the same receipt, a success-flag rule reports success while the graded rule reports unverified; (5) on a manual-mode line an approval-free binding's dispatch is intercepted into the queue, approval releasing all six parameters readback-verified; (6)–(7) API regression and documentation-consistency suites close the campaign.

Observed grade distribution. In fault-free operation, every batch dispatch of the six hardened runs closed *readback-verified*: structured approval released it and the three-way check read equal values at 0.0 bar maximum error (Fig. 4). The non-verified grades appear only under injected faults, the distribution expected under the specified grading rule: 1 *failed* (broker unreachable; healthy sibling verified), 1 *unverified* (publish-only endpoint, three attempts), and 2 parameters promoted from transport-acknowledged to readback-verified by post-write re-read. Transport-acknowledged occurs on MES-style endpoints with application-level acceptance but no readback mapping; MES route tests cover it, but the campaign archives MES in read mode only—a disclosed coverage gap.

Across the five questions, every exercised governed write produced an attributable evidence state. The window-edge probes and the publish-only receipt resolved as the grading rule predicts (Table II; Section V-G); the receipt-grading and master-switch surfaces were exercised in Q5, while Q1–Q4 evidence the path without them.

VI. Discussion and Limitations

A. Validity and Reproducibility

Seed, manifest, and raw artifacts are fixed; Q5 adds per-run ledgers and timelines. Five qualifications bound them: simulation-tier evidence (no physical-device campaign, operator study, or external comparison); internal-oracle optima and predicates under an author-implemented, author-adjudicated harness; single-host latencies and near-contemporary runs; unexercised capabilities and one live provider/model; and harness-adjudicated approval, with human latency and quality out of scope.

B. Safety Positioning and Human Oversight

The threat model is a misbehaving or manipulated agent inside an otherwise trusted platform and plant; approval, admission re-checks, and journaling are the containment surfaces, and no injection campaign or operator study is reported. Stolen credentials and compromised endpoints fall under IEC 62443 [18]; record-keeping and oversight duties belong to Regulation (EU) 2024/1689 [24], which the journal and approval gates serve at the evidence layer, certification out of scope. Protection functions remain in the PLC; prompts and retrieved documents are untrusted text, not authority [25], [26]. The proposer is confined to per-binding scope and journaled actions: a bounded blast radius, not a protection guarantee. Admission, grading, and the backstop run with or without human approval; enforcement never queues behind attention (deferred mode hands actuation to the approval policy, Section IV-D). Pacing bounds approval demand; batch release (Section V-G, leg 5) is the demonstrated fatigue mitigation, while approval fatigue and automation bias remain human-factors risks [23], [27], [28]. A lead agent's delegation-mode choice (Section IV-E) is the one agent-reachable path around per-request approval; capping it at the granter's binding mode or requiring line-switch confirmation is the queued guard, alongside the verdict-side guard of Section IV-D (independent adjudication or a platform-side re-check before `keep`).

C. Deployment Boundaries

Against the nearest systems of Section II, the differentiator is not the LLM—a replaceable strategy slot—but the production-scoped authority and evidence around its writes. The contribution is a framework claim—design, contracts, evidence semantics—assessed by construction, mechanism ablation, and two falsifiable contrasts (the window-edge probes

and the publish-only receipt), not by outcome superiority; an outcome-level comparison against SCADA-mediated operation is out of scope. The layer installs beside the plant's protocol endpoints, onboards declaratively (Section III-B), and leaves PLC protection and DCS interlocks untouched. Execution and recovery are non-atomic: bookkeeping may fail after actuation, pacing times writes rather than certifying stability, step limits are advisory, and multi-node compensation may partially succeed. Readback agreement and three-way consistency do not imply process recovery; hold heartbeats re-apply the setpoint without re-admission. Oscillation, storage failure, clock error, concurrency beyond four lines, and split-brain deployments remain unevaluated; the journal is append-only but not tamper-evident, so forensic integrity rests on host security and clock skew propagates into timelines. A supervisor outage halts dispatch; plant staff retain their DCS/SCADA stations, so the governed path mediates agent-originated and platform-client writes, not existing operator tooling—an integration boundary, not a takeover. The next evidence tier is a hardware-in-the-loop leg against a real PLC or OPC-UA endpoint and community benchmarks such as Tennessee Eastman [29]. In production, bindings govern sanctioned tools, not arbitrary privileged-engine I/O; least privilege, endpoint security, and IT/OT separation remain necessary [18]. The platform is neither a safety instrumented function nor a plant-protection substitute [30], and line-scoped approver authorization is still missing, so production needs segregation of duties and alarm-path failure tests.

VII. Conclusion

The platform contributes a governed supervisory execution layer, not a new optimizer: one path from agent-team identities to governed writes, graded acknowledgment, verdict, and compensation, with admission, approval, and recovery left as separately inspectable, actor-attributed evidence on every agent-originated write—the actionable unit, independently inspectable and revocable. Physical-plant deployment, operator studies, and external comparison remain open; optimization benefit, autonomous control, and safety assurance stay unsupported claims.

Data availability and disclosure. Artifacts are archived per run; the platform and simulators are publicly available (links withheld for double-blind review; Section III), the live engine-turn transcripts survive only as recorded outcomes, and the write-evidence suite is rerunnable. AI assistants supported editing under author direction; the authors remain responsible for all claims.

References

[1] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in *Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD)*, 2026, pp. 1–6.

[2] K. Köhle, D. Anicic, T. A. Runkler, and R. Graf, "LLM-driven multi-agent control for skill-based smart manufacturing," in *Proc. IEEE 31st Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2026.

[3] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in *Proc. IEEE 21st Int. Conf. on Automation Science and Engineering (CASE)*, 2025.

[4] Z. Zhao *et al.*, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888.

[5] M. Waseem, K. Bhatta, C. Li, and Q. Chang, "Pretrained LLMs as real-time controllers for robot operated serial production line," *Mach. Learn.: Eng.*, vol. 2, no. 1, 2026, art. no. 015006.

[6] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber–physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025.

[7] Z. Liu *et al.*, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026.

[8] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. 2025 IEEE 64th Conference on Decision and Control (CDC)*, 2025, pp. 172–179.

[9] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, pp. 1–8.

[10] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025.

[11] X. Jiang, H. Xie, J. Wang, Z. Yang, Y. Zhou, L. Yao, and Z. Zhu, "Agentic AI for safety-aware process monitoring and fault diagnosis: A review," *Processes*, vol. 14, 2026, art. no. 2112.

[12] Anthropic, "Model context protocol," [Online]. Available: https://modelcontextprotocol.io, 2024, accessed: Sep. 20, 2026.

[13] ISA, "ANSI/ISA-88.00.01-2010—batch control—part 1: Models and terminology; ANSI/ISA-95.00.01-2010—enterprise-control system integration—part 1: Models and terminology," Standard, 2010.

[14] J. Moyne, E. del Castillo, and A. M. Hurwitz, *Run-to-Run Control in Semiconductor Manufacturing*. Boca Raton, FL: CRC Press, 2000.

[15] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," 2025, arXiv preprint arXiv:2503.18666.

[16] E. Debenedetti *et al.*, "Defeating prompt injections by design," 2025, arXiv preprint arXiv:2503.18813.

[17] U.S. Food and Drug Administration, "21 CFR Part 11: Electronic records; electronic signatures—scope and application," Regulation guidance, 2003.

[18] IEC, "IEC 62443-3-3:2013: Industrial communication networks—network and system security—part 3-3: System security requirements and security levels," Standard, 2013.

[19] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.

[20] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, 2017.

[21] L. Sha, "Using simplicity to control complexity," *IEEE Softw.*, vol. 18, no. 4, pp. 20–28, 2001.

[22] IEEE, "IEEE std 1815-2012: Standard for electric power systems communications—distributed network protocol (DNP3)," Standard, 2012.

[23] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983.

[24] European Union, "Regulation (EU) 2024/1689 of the european parliament and of the council laying down harmonised rules on artificial intelligence (artificial intelligence act)," Official Journal of the European Union, 2024.

[25] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90.

[26] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1831–1847.

[27] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010.

[28] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000.

[29] J. J. Downs and E. F. Vogel, "A plant-wide industrial process control problem," *Comput. Chem. Eng.*, vol. 17, no. 3, pp. 245–255, 1993.

[30] IEC, "IEC 61511-1:2016: Functional safety—safety instrumented systems for the process industry sector," 2016.